30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 7)

“Hands-on Mini Project - Blinking Two LEDs with Two Tasks”



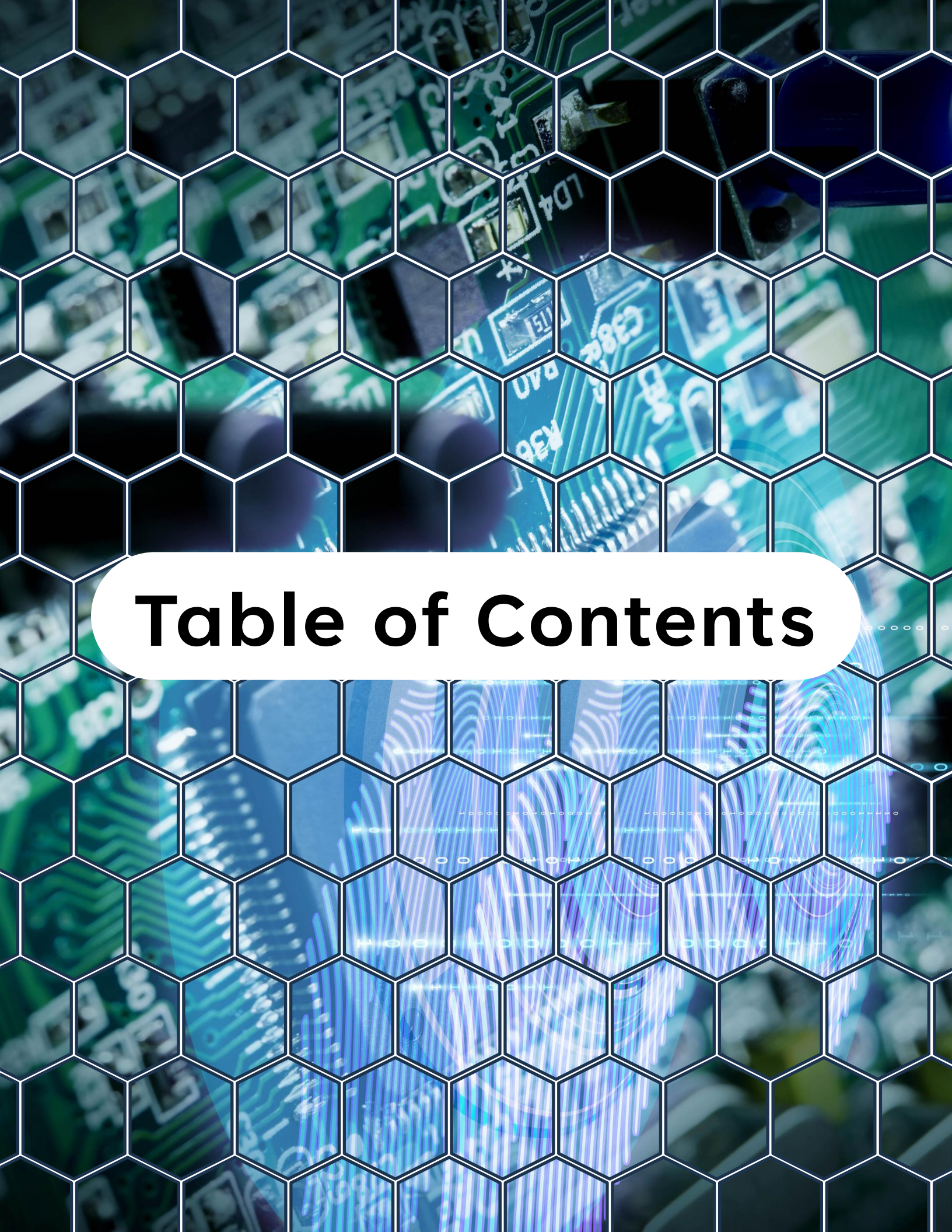
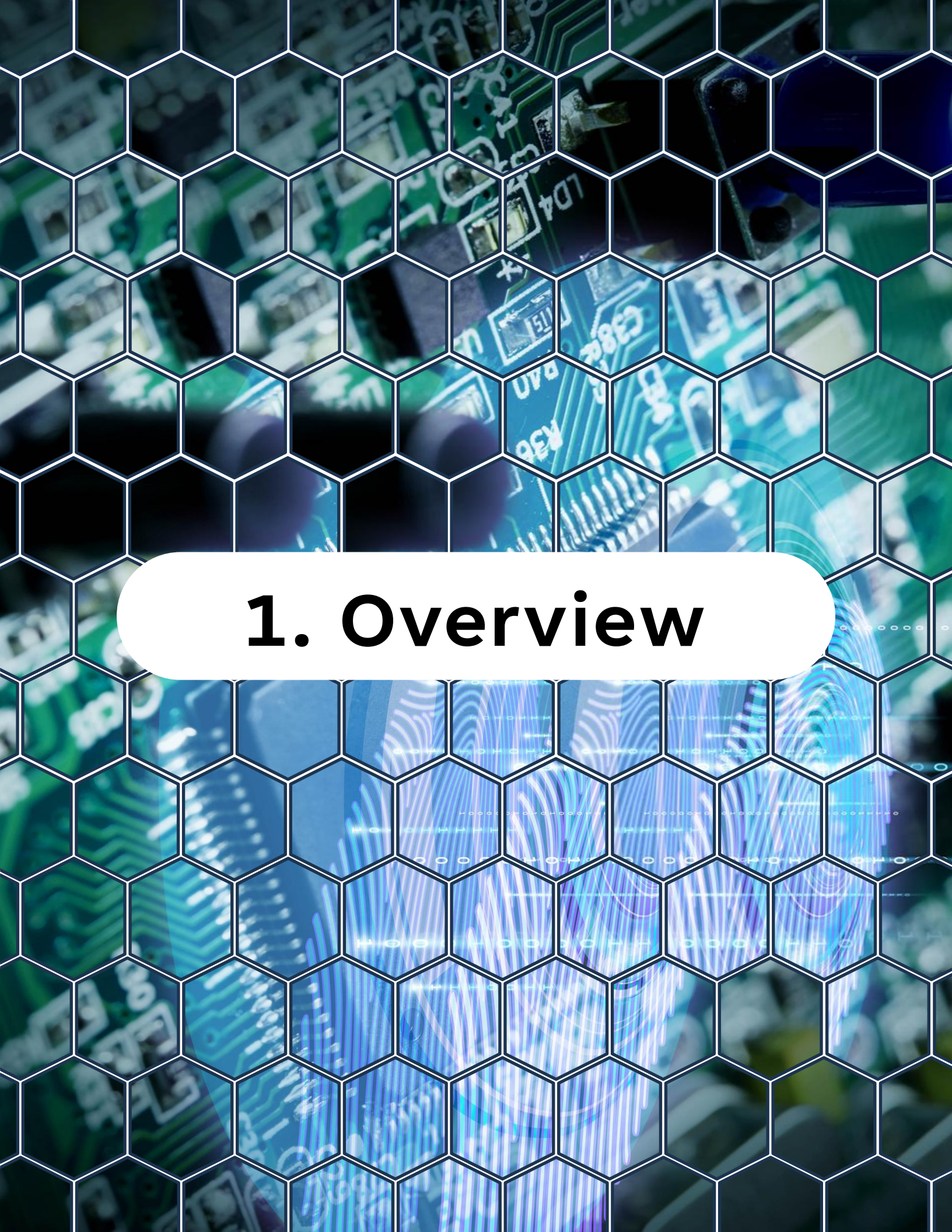


Table of Contents

Table of Contents

1. Overview
2. Hardware Setup
3. Full Source Code (ESP-IDF + FreeRTOS)
4. What to Observe
5. Troubleshooting & Tips
6. Exercises (Optional Enhancements)
7. Summary

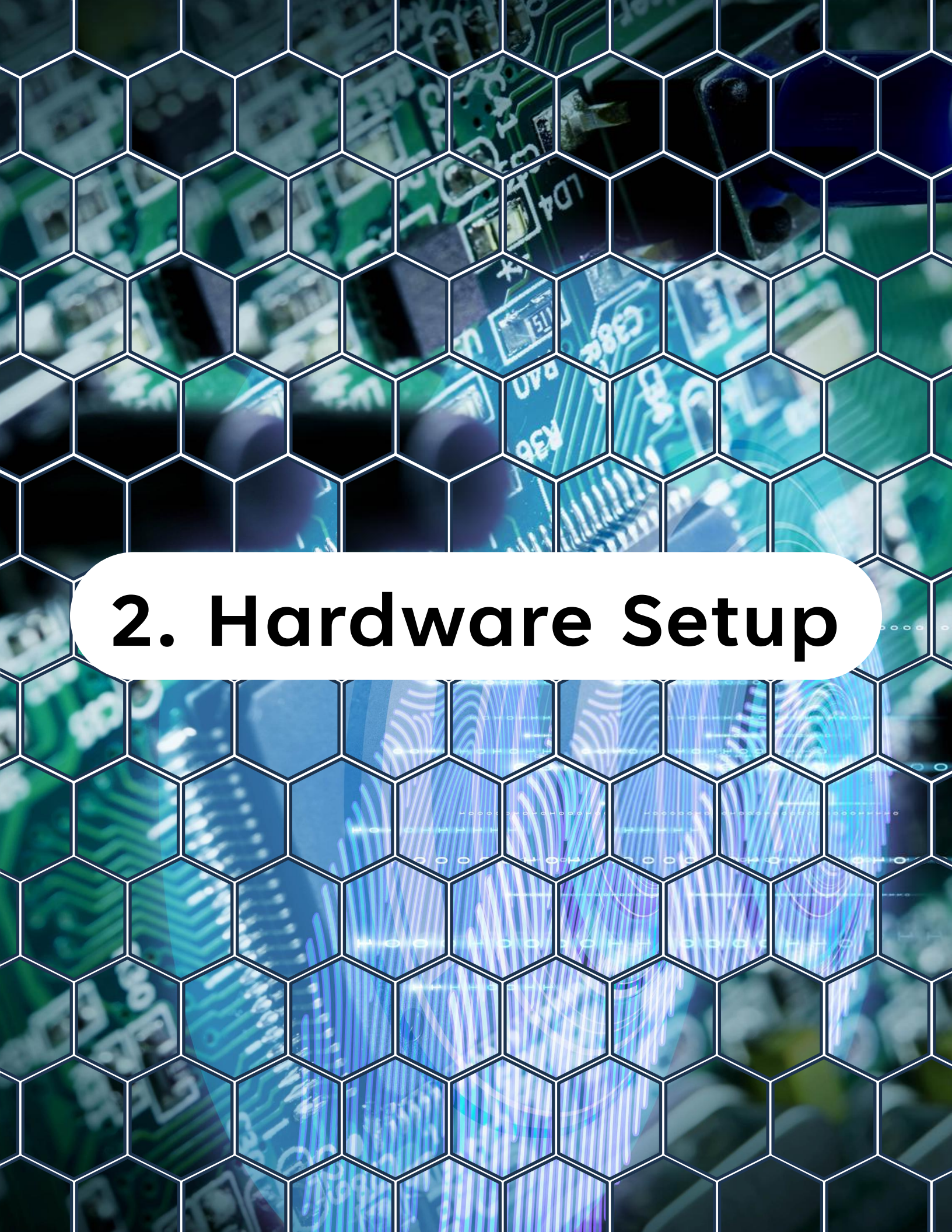


1. Overview

1. Overview

On **Day 7**, you'll learn:

- Wire two LEDs to your ESP32.
- Create **two FreeRTOS tasks**:
  - Task A (uses `vTaskDelay()`) → LED1 @ 500 ms
  - Task B (uses `vTaskDelayUntil()`) → LED2 @ 1000 ms (fixed cadence)
- Observe timing drift vs. steady period.
- Measure drift using the tick counter.
- Learn simple patterns for GPIO setup and periodic tasks.



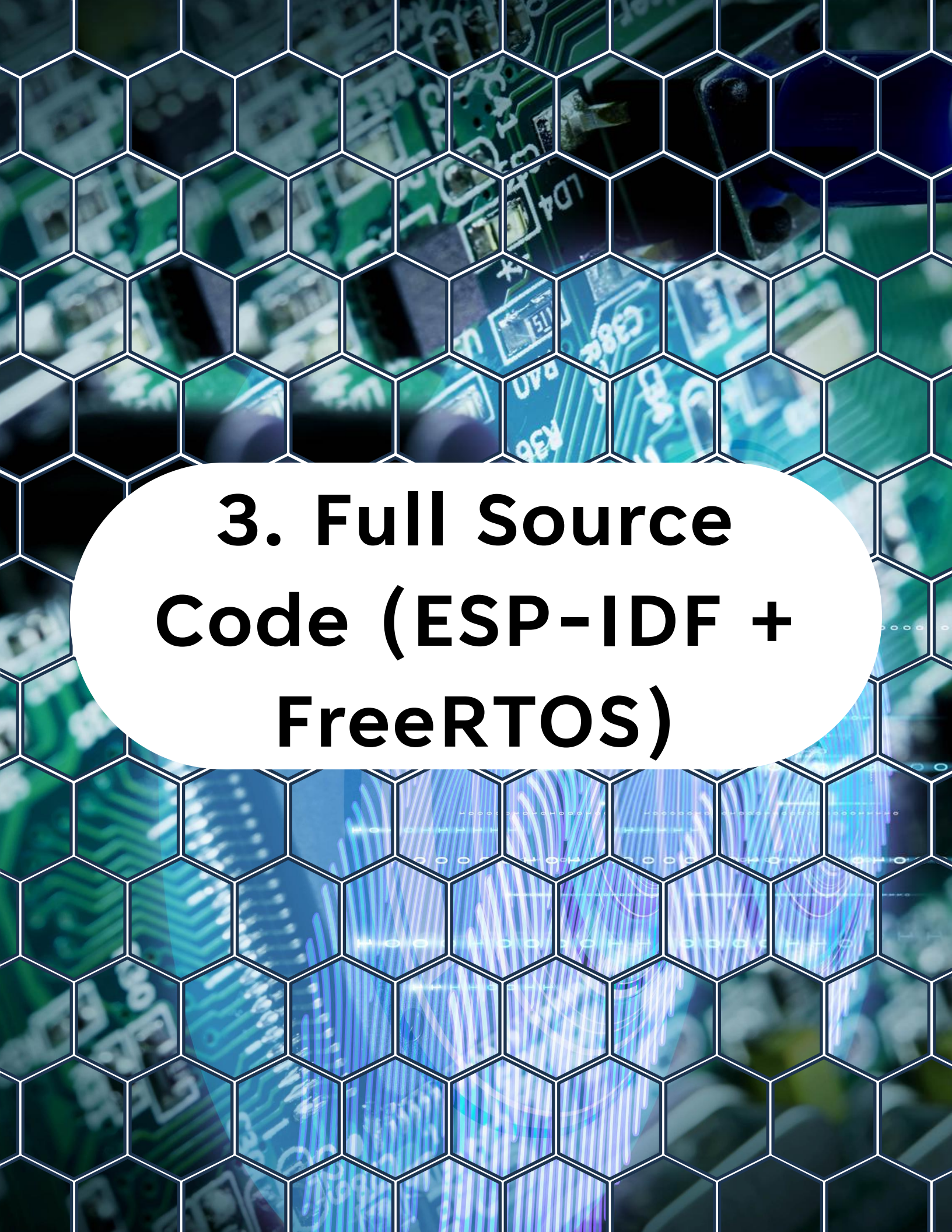
2. Hardware Setup

2. Hardware Setup

- **LED1** anode → **GPIO 2** (default; adjust for your board)
- **LED2** anode → **GPIO 4**
- Each LED cathode → series resistor (220–330 Ω) → **GND**

Boards differ:

- **Many ESP32 DevKit boards expose GPIO2 with an onboard LED.**
- **If your board routes these pins elsewhere, change the LED1_GPIO / LED2_GPIO defines accordingly.**



3. Full Source Code (ESP-IDF + FreeRTOS)

3. Full Source Code (ESP-IDF + FreeRTOS)

```c
/*
 * Day 7 – Hands-on Mini Project: Blinking Two LEDs with Two Tasks
 *
 * This example demonstrates timing differences between
 * vTaskDelay() (relative delay) and vTaskDelayUntil() (absolute
 * periodic schedule). Two LEDs blink at
 * different rates:
 *  - Task A: LED1 toggles every 500 ms using vTaskDelay()
 *            -> may drift over long runs
 *  - Task B: LED2 toggles every 1000 ms using vTaskDelayUntil()
 *            -> stays on a fixed cadence
 *
 * Wiring:
 *   LED1 (GPIO2 by default) -> resistor -> GND
 *   LED2 (GPIO4 by default) -> resistor -> GND
 *
 * Notes:
 *   - Adjust LED1_GPIO / LED2_GPIO for your hardware if needed.
 *   - Prints drift information to the console to illustrate timing behavior.
 */

#include <stdio.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"

#define TAG "DAY7"

// === Adjust these for your board if necessary ===
#ifndef LED1_GPIO
#define LED1_GPIO GPIO_NUM_2   // Often has onboard LED on many DevKit boards
#endif

#ifndef LED2_GPIO
#define LED2_GPIO GPIO_NUM_4
#endif

static inline void configure_led(gpio_num_t pin)
{
    // Configure pin as output and start low
```

3. Full Source Code (ESP-IDF + FreeRTOS)

```c
static inline void configure_led(gpio_num_t pin)
{
    // Configure pin as output and start low
    gpio_config_t io = {
        .pin_bit_mask = (1ULL << pin),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&io);
    gpio_set_level(pin, 0);
}

static inline void toggle_led(gpio_num_t pin)
{
    int current = gpio_get_level(pin);
    gpio_set_level(pin, !current);
}

/*
 * Task A: Blink LED1 using vTaskDelay() (relative delay).
 * Demonstrates drift accumulation when the loop body takes non-zero time.
 */
static void taskA_delay(void *pv)
{
    const TickType_t period_ticks = pdMS_TO_TICKS(500); // 500 ms
    uint32_t iteration = 0;

    ESP_LOGI(TAG, "Task A (vTaskDelay) started on core %d", xPortGetCoreID());

    while (1) {
        toggle_led(LED1_GPIO);
        iteration++;

        // Print a timestamp (ms) to visualize drift
        TickType_t now_ticks = xTaskGetTickCount();
        uint32_t now_ms = now_ticks * portTICK_PERIOD_MS;
        ESP_LOGI(TAG, "[A] t=%" PRIu32 " ms, iter=%" PRIu32, now_ms, iteration);

        // Relative delay: next wake-up occurs 'period_ticks' after this call
        vTaskDelay(period_ticks);
    }
}
```

3. Full Source Code (ESP-IDF + FreeRTOS)

```c
/*
 * Task B: Blink LED2 using vTaskDelayUntil() (absolute periodic schedule).
 * Keeps a stable cadence referenced to the first captured tick.
 */
static void taskB_delay_until(void *pv)
{
    const TickType_t period_ticks = pdMS_TO_TICKS(1000); // 1000 ms
    TickType_t last_wake = xTaskGetTickCount();          // Reference (t0)
    uint32_t iteration = 0;

    ESP_LOGI(TAG, "Task B (vTaskDelayUntil) started on core %d", xPortGetCoreID());

    while (1) {
        toggle_led(LED2_GPIO);
        iteration++;

        TickType_t now_ticks = xTaskGetTickCount();
        uint32_t now_ms = now_ticks * portTICK_PERIOD_MS;
        ESP_LOGI(TAG, "[B] t=%" PRIu32 " ms, iter=%" PRIu32, now_ms, iteration);

        // Absolute delay: wake exactly every 'period_ticks' since last_wake
        vTaskDelayUntil(&last_wake, period_ticks);
    }
}

/*
 * Optional status task:
 * Prints how far Task A has "slipped" from an ideal schedule
 * compared to an ideal 500 ms cadence. This is a didactic metric, not a strict
 * real-time analysis tool.
 */
static void task_status(void *pv)
{
    const TickType_t period_ticks = pdMS_TO_TICKS(2000); // every 2 s
    TickType_t last_wake = xTaskGetTickCount();
    uint32_t seconds = 0;

    ESP_LOGI(TAG, "Status task started on core %d", xPortGetCoreID());

    while (1) {
        vTaskDelayUntil(&last_wake, period_ticks);
        seconds += 2;

        // Report uptime and tick count
```

3. Full Source Code (ESP-IDF + FreeRTOS)

```
 * Optional status task:
 * Prints how far Task A has "slipped" from an ideal schedule
 * compared to an ideal 500 ms cadence. This is a didactic metric, not a strict
 * real-time analysis tool.
 */
static void task_status(void *pv)
{
    const TickType_t period_ticks = pdMS_TO_TICKS(2000); // every 2 s
    TickType_t last_wake = xTaskGetTickCount();
    uint32_t seconds = 0;

    ESP_LOGI(TAG, "Status task started on core %d", xPortGetCoreID());

    while (1) {
        vTaskDelayUntil(&last_wake, period_ticks);
        seconds += 2;

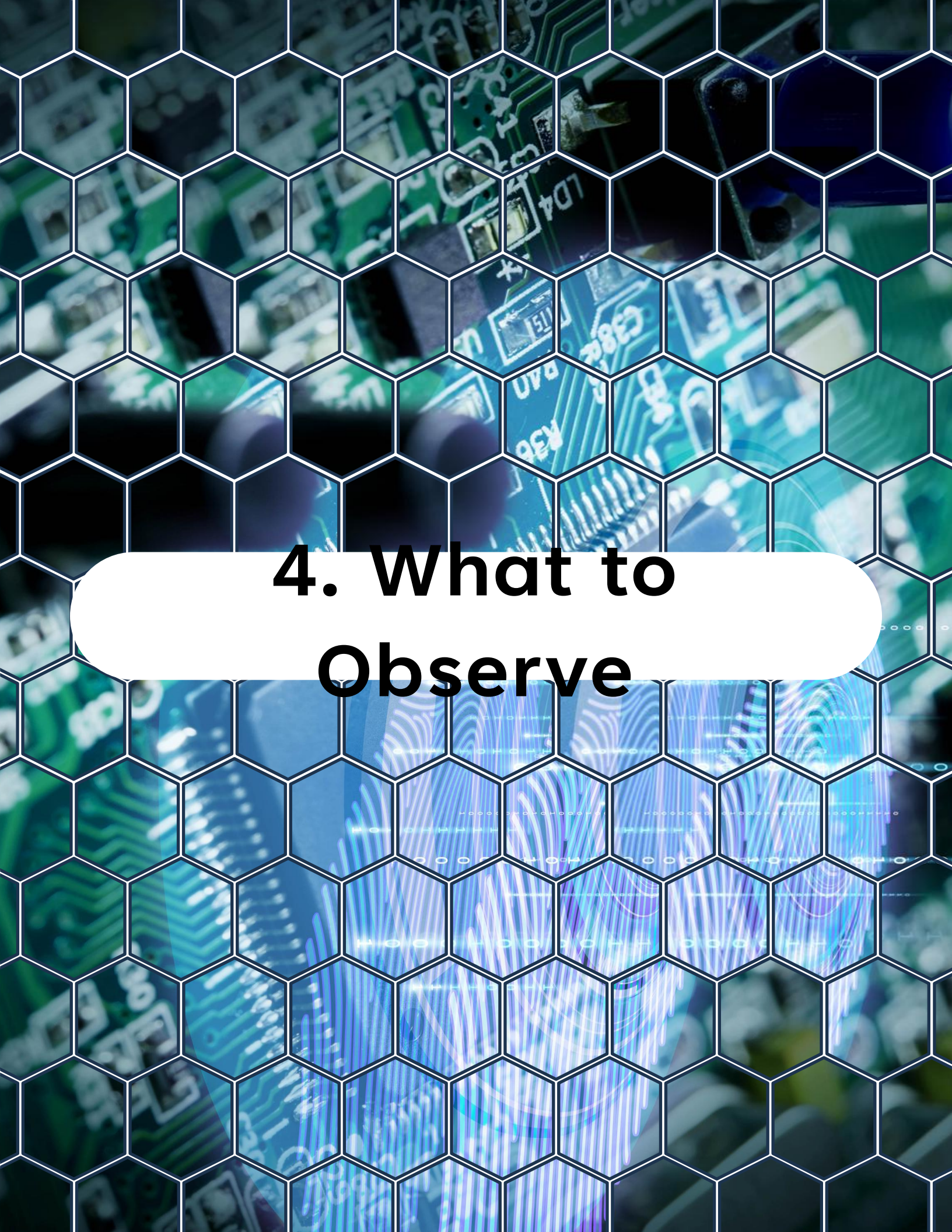
        // Report uptime and tick count
        TickType_t ticks = xTaskGetTickCount();
        uint32_t ms = ticks * portTICK_PERIOD_MS;
        ESP_LOGI(TAG, "[STATUS] uptime ~%" PRIu32 " ms (%" PRIu32 "s)", ms, seconds);
    }
}

void app_main(void)
{
    // --- GPIO setup
    configure_led(LED1_GPIO);
    configure_led(LED2_GPIO);

    // --- Create tasks
    // Priorities kept the same to let the scheduler time-slice fairly
    BaseType_t okA = xTaskCreate(taskA_delay, "TaskA_Delay", 2048, NULL, 5, NULL);
    BaseType_t okB = xTaskCreate(taskB_delay_until, "TaskB_DelayUntil",
                                 2048, NULL, 5, NULL);

    // Optional status task (lower priority)
    xTaskCreate(task_status, "TaskStatus", 2048, NULL, 3, NULL);

    if (okA != pdPASS || okB != pdPASS) {
        ESP_LOGE(TAG, "Failed to create one or more tasks");
    }
}
```



4. What to Observe

4. What to Observe

Open the serial monitor (`idf.py monitor` or the ESP-IDF Plugin `Monitor Device` option):

- Lines prefixed [**A**]: timestamps for the `vTaskDelay()` task.

  You'll notice that execution time inside the loop adds on top of the delay, causing **slight drift** over time.
- Lines prefixed [**B**]: timestamps for the `vTaskDelayUntil()` task.

  Wake-ups occur at a **fixed cadence** (every 1000 ms), maintaining alignment with the initial reference tick (as long as the loop finishes before the next interval).

4. What to Observe

Why This Matters

`vTaskDelay()` is simple and fine for non-critical timing, but your period becomes "**delay + work time**".

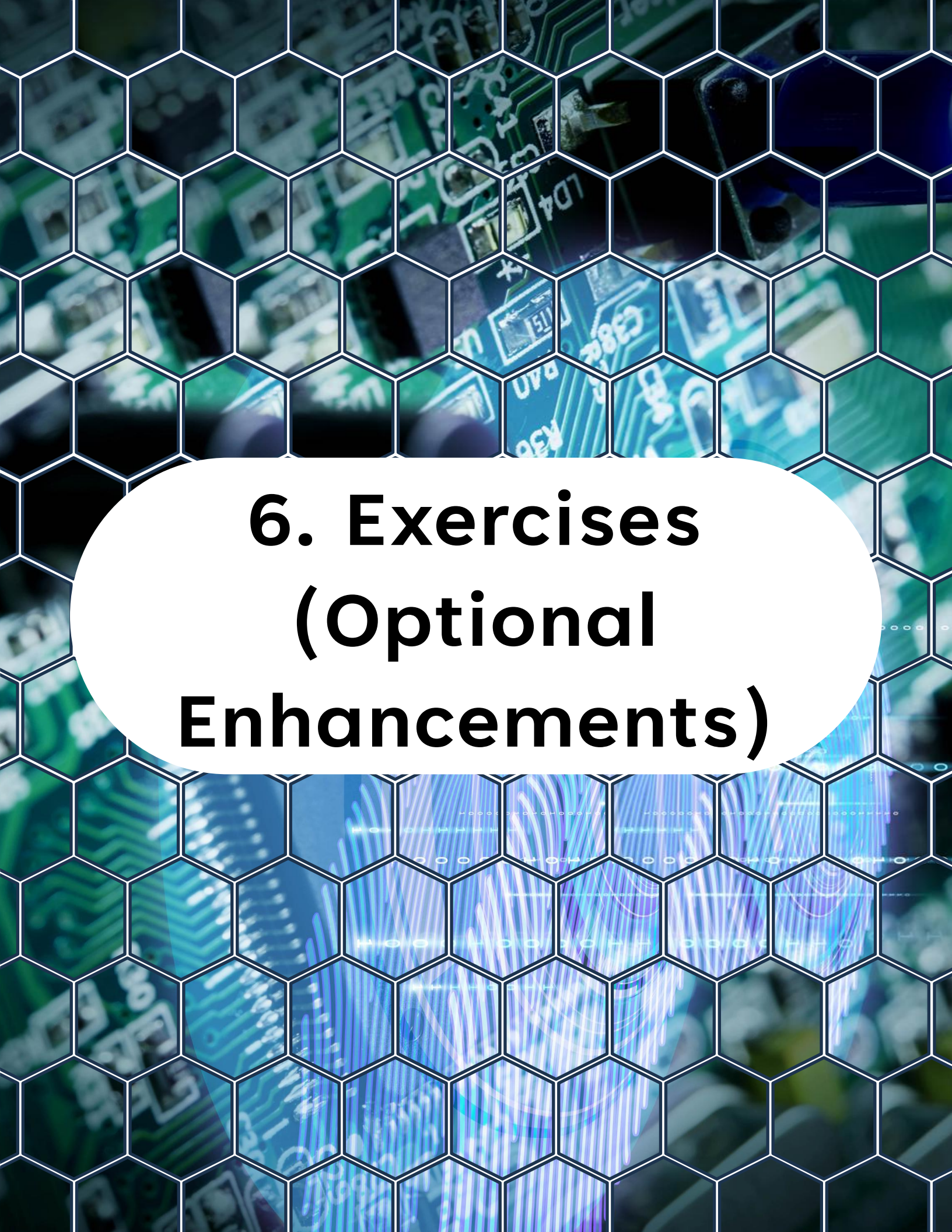
`vTaskDelayUntil()` creates **true periodic tasks** by scheduling the next wake-up at an **absolute tick**, greatly reducing cumulative drift.



5. Troubleshooting & Tips

5. Troubleshooting & Tips

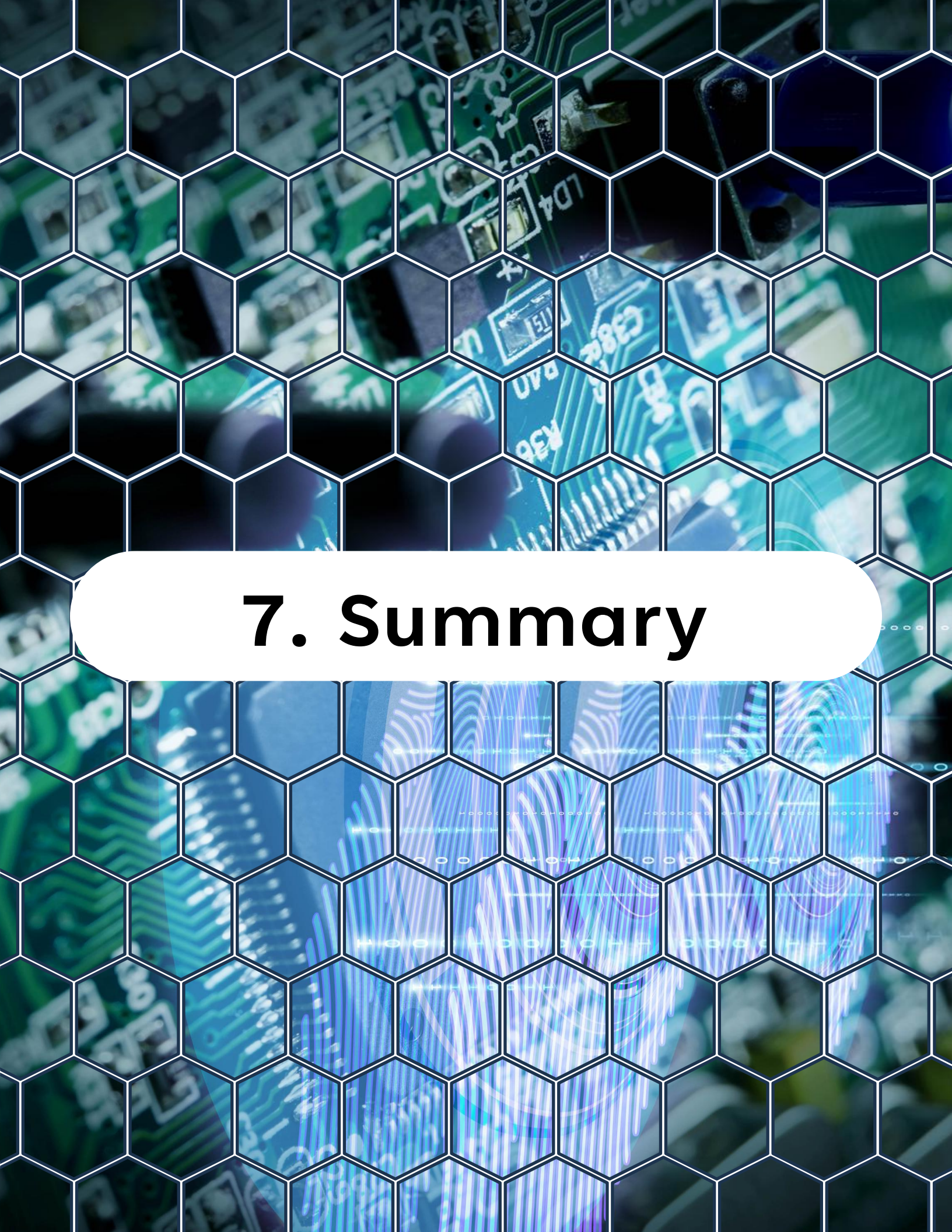
- If you don’t see LED activity, verify your **GPIO numbers** and **wiring**.
- Some boards drive onboard LEDs **active-low**. If your LED seems inverted, swap the polarity or invert the logic.
- If logs are too verbose, adjust the logging level in **menuconfig → Component config → Log output** or replace **ESP_LOGI** with **printf**.
- Keep each task’s loop work **shorter than its period**. If a loop iteration exceeds the period, `vTaskDelayUntil()` will “catch up” (wake immediately), and you’ll see warning-level jitter.



6. Exercises (Optional Enhancements)

6. Exercises (Optional Enhancements)

1. **Core Affinity:** Pin Task A to Core 0 and Task B to Core 1 with xTaskCreatePinnedToCore(). Observe any differences.
2. **Button Pause:** Add a GPIO button that toggles pausing Task A using vTaskSuspend() / vTaskResume().
3. **Timing Stress:** Add a busy loop of 5–10 ms in Task A and watch drift increase vs. Task B's stability.
4. **Precision:** Raise the tick rate to 1000 Hz in menuconfig (1 ms tick) and compare jitter.
5. **Mutexed Logger:** Wrap logging with a mutex so only one task prints at a time (teaser for Day 12).



7. Summary

7. Summary

- Built two LED tasks to **compare relative** vs. **absolute delays**.
- Observed **drift** with `vTaskDelay()` and steady **cadence** with `vTaskDelayUntil()`.
- Learned a clean pattern for **GPIO setup** and **periodic task structure**.

"Thanks for watching!

If you

enjoyed this video,

make sure to hit the like button and

subscribe to stay updated with my latest

content.

Don't forget to check out my other videos

for more tips and tutorials on Embedded C,

Python, hardware designs, etc. Keep

exploring, keep learning, and I'll see you in

the next

video!"



https://www.linkedin.com/in/yamil-garcia
https://www.youtube.com/@LearningByTutorials
https://github.com/god233012yamil